

State Prop Drilling SPA Let Const Var UseEffect

Aamir Hussain Azad

State Prop Drilling SPA Let Const Var UseEffect

Here's a **complete, structured explanation of ALL React concepts** from fundamentals to advanced, organized for easy learning.



COMPLETE REACT CONCEPT GUIDE

TABLE OF CONTENTS

1. Core Fundamentals
2. Components & Props
3. State Management
4. Hooks (Comprehensive)
5. Rendering & Performance
6. Context API
7. Refs & DOM
8. Side Effects & Lifecycle
9. Advanced Patterns
10. React 18+ Features
11. Ecosystem & Tools

1. CORE FUNDAMENTALS

What is React?

React is a JavaScript library for building user interfaces, developed by Facebook.

Key Characteristics:

- **Declarative:** Describe *what* UI should look like, React handles *how* to update

- **Component-Based:** Encapsulated components manage their own state
- **Learn Once, Write Anywhere:** React DOM, React Native, React VR

Virtual DOM

javascript

```
const vDOM = {  
  type: 'div',  
  props: { className: 'container' },  
  children: ['Hello World']  
};
```

JSX (JavaScript XML)

jsx

```
const element = <h1 className="greeting">Hello, {name}!</h1>;  
  
const element = React.createElement('h1',  
  { className: 'greeting' },  
  `Hello, ${name}!`  
);
```

JSX Rules:

- Must have one parent element (use Fragment `<> </>`)
- Use `className` instead of `class`
- Use `htmlFor` instead of `for`
- Self-closing tags: `<img />`, `<input />`

- Embed JavaScript in `{}`

2. COMPONENTS & PROPS

Functional Components (Modern)

jsx

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}  
  
const Welcome = ({ name, age }) => (  
  <div>  
    <h1>Hello, {name}</h1>  
    <p>Age: {age}</p>  
  </div>  
);
```

Class Components (Legacy)

jsx

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

Props (Properties)

Read-only data passed from parent to child

jsx

```
function Parent() {
  return (
    <Child
      name="John"
      age={25}
      isActive={true}
      items={['a', 'b']}
      onCustomClick={handleClick}
    />
  );
}

function Child({ name, age, isActive, items, onCustomClick })
{
  return (
    <div onClick={onCustomClick}>
      {name} is {age} years old
    </div>
  );
}

Child.defaultProps = {
  isActive: false,
  items: []
};

import PropTypes from 'prop-types';
Child.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number,
  onCustomClick: PropTypes.func
};
```

Children Prop

jsx

```
function Card({ children, title }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="content">{children}</div>
    </div>
  );
}

<Card title="My Card">
  <p>This is child content</p>
  <button>Click me</button>
</Card>
```

Component Composition

jsx

```
function Layout({ sidebar, content }) {
  return (
    <div className="layout">
      <div className="sidebar">{sidebar}</div>
      <div className="content">{content}</div>
    </div>
  );
}

function Dialog({ title, message, variant }) {
  return (
    <div className={`dialog dialog-${variant}`}>
      <h2>{title}</h2>
      <p>{message}</p>
    </div>
  );
}

function WarningDialog({ message }) {
  return <Dialog title="Warning" message={message}
    variant="warning" />;
}
```

3. STATE MANAGEMENT

useState Hook

jsx

```
const [state, setState] = useState(initialValue);

const [count, setCount] = useState(0);
const [user, setUser] = useState(null);
const [todos, setTodos] = useState([]);
const [form, setForm] = useState({ name: '', email: '' });

setCount(count + 1);

setCount(prevCount => prevCount + 1);

setUser({ ...user, name: 'John' });

setTodos([...todos, newTodo]);
setTodos(todos.filter(todo => todo.id !== id));
```

State Management Patterns

Lifting State Up

jsx

```
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <>
      <ChildA count={count} />
      <ChildB onIncrement={() => setCount(count + 1)} />
    </>
  );
}
```

State Colocation

jsx

```
function App() {  
  return <Footer />;  
}  
  
function Footer() {  
  const [theme, setTheme] = useState('light');  
  return <div>Theme: {theme}</div>;  
}
```

useReducer Hook

For complex state logic with multiple sub-values or transitions

jsx

```
const initialState = { count: 0, step: 1 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { ...state, count: state.count + state.step };
    case 'decrement':
      return { ...state, count: state.count - state.step };
    case 'setStep':
      return { ...state, step: action.payload };
    case 'reset':
      return initialState;
    default:
      return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <>
      Count: {state.count}
      <button onClick={() => dispatch({ type: 'increment'
    })}></button>
      <button onClick={() => dispatch({ type: 'decrement'
    })}></button>
      <input
        value={state.step}
        onChange={e => dispatch({
          type: 'setStep',
          payload: Number(e.target.value)
        })}
      />
    </>
  );
}
```


4. HOOKS (COMPREHENSIVE)

Basic Hooks

useState (covered above)

useEffect (detailed in section 8)

useContext (covered in section 6)

Additional Hooks

useRef

jsx

```
function TextInput() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <>
      <input ref={inputRef} type="text" />
      <button onClick={focusInput}>Focus</button>
    </>
  );
}

function Timer() {
  const intervalRef = useRef();
  const [count, setCount] = useState(0);

  useEffect(() => {
    intervalRef.current = setInterval(() => {
      setCount(c => c + 1);
    }, 1000);
    return () => clearInterval(intervalRef.current);
  }, []);

  return <div>{count}</div>;
}
```

useMemo

Memoizes expensive calculations

jsx

```
function ExpensiveComponent({ numbers, filter }) {

  const filteredNumbers = useMemo(() => {
    console.log('Filtering numbers...');
    return numbers.filter(n => n > filter);
  }, [numbers, filter]);

  return <div>{filteredNumbers.join(', ')}</div>;
}

function Fibonacci({ n }) {
  const fibValue = useMemo(() => {
    const fib = (num) => {
      if (num <= 1) return num;
      return fib(num - 1) + fib(num - 2);
    };
    return fib(n);
  }, [n]);

  return <div>Fibonacci({n}) = {fibValue}</div>;
}
```

useCallback

Memoizes functions to prevent unnecessary re-renders

jsx

```
function Parent() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState('');

  const handleClick = useCallback(() => {
    setCount(c => c + 1);
  }, []);

  const handleSave = useCallback((data) => {
    console.log('Saving:', data, count);
  }, [count]);

  return (
    <>
      <input value={text} onChange={e =>
setText(e.target.value)} />
      <Child onClick={handleClick} onSave={handleSave} />
    </>
  );
}

const Child = React.memo(({ onClick, onSave }) => {

  return <button onClick={onClick}>Click</button>;
});
```

useImperativeHandle

Customize ref handle exposed to parent

jsx

```
const FancyInput = forwardRef((props, ref) => {
  const inputRef = useRef();

  useImperativeHandle(ref, () => ({
    focus: () => {
      inputRef.current.focus();
    },
    clear: () => {
      inputRef.current.value = '';
    },
    getValue: () => inputRef.current.value
  }));

  return <input ref={inputRef} {...props} />;
});

function Parent() {
  const ref = useRef();

  return (
    <>
      <FancyInput ref={ref} />
      <button onClick={() =>
        ref.current.focus() }>Focus</button>
      <button onClick={() =>
        ref.current.clear() }>Clear</button>
    </>
  );
}
```

useLayoutEffect

Runs synchronously after DOM mutations (before paint)

jsx

```
function MeasureComponent() {  
  const [height, setHeight] = useState(0);  
  const ref = useRef();  
  
  useEffect(() => {  
  
    setHeight(ref.current.clientHeight);  
  }, []);  
  
  return <div ref={ref}>Measure my height: {height}px</div>;  
}
```

useDebugValue

Label custom hooks in React DevTools

jsx

```
function useFriendStatus(friendID) {  
  const [isOnline, setIsOnline] = useState(null);  
  
  useDebugValue(isOnline ? 'Online' : 'Offline');  
  useDebugValue(friendID, id => `Friend: ${id}`);  
  
  return isOnline;  
}
```

useDeferredValue (React 18)

Defer non-urgent updates

jsx

```
function SearchResults({ query }) {
  const deferredQuery = useDeferredValue(query);
  const results = useMemo(() => {
    return searchItems(deferredQuery);
  }, [deferredQuery]);

  return (
    <div style={{ opacity: query !== deferredQuery ? 0.5 : 1
  }}>
      {results.map(item => <div key={item.id}>{item.name}
</div>)}
    </div>
  );
}
```

useTransition (React 18)

Mark updates as non-urgent

jsx

```
function App() {
  const [isPending, startTransition] = useTransition();
  const [tab, setTab] = useState('about');

  const switchTab = (newTab) => {
    startTransition(() => {
      setTab(newTab);
    });
  };

  return (
    <div>
      {isPending && <Spinner />}
      <button onClick={() =>
switchTab('about')}>About</button>
      <button onClick={() =>
switchTab('contact')}>Contact</button>
      {tab === 'about' && <About />}
      {tab === 'contact' && <Contact />}
    </div>
  );
}
```

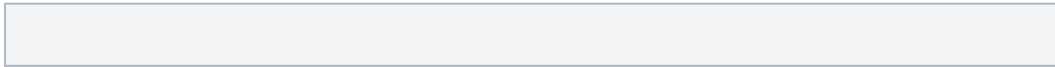
5. RENDERING & PERFORMANCE

Reconciliation & Diffing Algorithm

How React decides what to update:

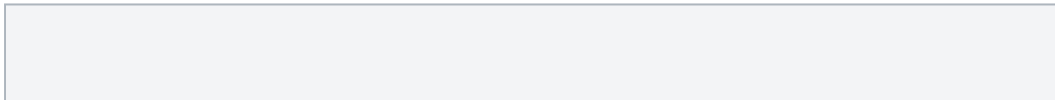
1. **Different element types** → Destroy and rebuild

jsx



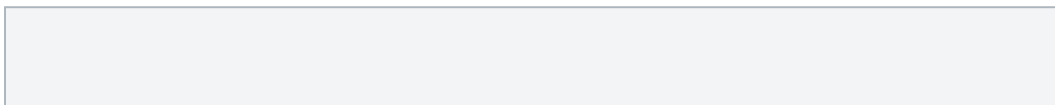
2. **Same element type** → Update attributes

jsx



3. **Recurse through children** → Use keys

jsx



Keys in Lists

jsx

```
{todos.map((todo, index) => (  
  <Todo key={index} {...todo} />  
))}  
  
{todos.map(todo => (  
  <Todo key={todo.id} {...todo} />  
))}  
  
{todos.map((todo, index) => (  
  <Todo key={`_${todo.text}-${index}`} {...todo} />  
))}
```

Rendering Optimizations

React.memo

jsx

```
const Child = React.memo(({ data }) => {  
  console.log('Child rendered');  
  return <div>{data}</div>;  
});
```

useMemo & useCallback (covered above)

Code Splitting with React.lazy

jsx

```
import { lazy, Suspense } from 'react';  
  
const HeavyComponent = lazy(() => import('./HeavyComponent'));  
  
function App() {  
  return (  
    <Suspense fallback={<div>Loading...</div>}>  
      <HeavyComponent />  
    </Suspense>  
  );  
}
```

Virtualization (react-window)

jsx


```
import { FixedSizeList as List } from 'react-window';

function VirtualList({ items }) {
  const Row = ({ index, style }) => (
    <div style={style}>
      {items[index]}
    </div>
  );

  return (
    <List
      height={400}
      itemCount={items.length}
      itemSize={35}
      width={300}
    >
      {Row}
    </List>
  );
}
```

Avoid Inline Functions in Render

jsx

```
<button onClick={() => handleClick(id)}>Click</button>

const handleButtonClick = useCallback(() => {
  handleClick(id);
}, [id]);

<button onClick={handleButtonClick}>Click</button>
```

6. CONTEXT API

Creating and Using Context

jsx

```
const ThemeContext = React.createContext('light');
const UserContext = React.createContext();

function App() {
  const [theme, setTheme] = useState('light');
  const [user, setUser] = useState(null);

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <UserContext.Provider value={user}>
        <Dashboard />
      </UserContext.Provider>
    </ThemeContext.Provider>
  );
}

function Dashboard() {
  const { theme, setTheme } = useContext(ThemeContext);
  const user = useContext(UserContext);

  return (
    <div className={`app ${theme}`}>
      <button onClick={() => setTheme('dark')}>
        Switch to Dark
      </button>
      <h1>Welcome, {user?.name}</h1>
    </div>
  );
}
```

Context with useReducer (Global State)

jsx

```
const AppContext = React.createContext();

const initialState = {
  user: null,
  theme: 'light',
  notifications: []
};

function appReducer(state, action) {
  switch (action.type) {
    case 'SET_USER':
      return { ...state, user: action.payload };
    case 'SET_THEME':
      return { ...state, theme: action.payload };
    case 'ADD_NOTIFICATION':
      return {
        ...state,
        notifications: [...state.notifications,
action.payload]
      };
    default:
      return state;
  }
}

function AppProvider({ children }) {
  const [state, dispatch] = useReducer(appReducer,
initialState);

  return (
    <AppContext.Provider value={{ state, dispatch }}>
      {children}
    </AppContext.Provider>
  );
}

function LoginButton() {
  const { dispatch } = useContext(AppContext);

  const login = () => {
    dispatch({ type: 'SET_USER', payload: { name: 'John' } });
  };

  return <button onClick={login}>Login</button>;
}
```

Context Performance Optimizations

jsx

```
<MyContext.Provider value={{ user, theme, setTheme }}>

// ✅ Good: Split contexts by usage
const UserContext = React.createContext();
const ThemeContext = React.createContext();

// ✅ Even better: Memoize context value
const memoizedValue = useMemo(() => ({ user, setUser }),
[user]);
<UserContext.Provider value={memoizedValue}>
```

7. REFS & DOM

Creating Refs

jsx

```
const inputRef = useRef(null);

const measureRef = useCallback(node => {
  if (node) {
    const rect = node.getBoundingClientRect();
    console.log(rect.width);
  }
}, []);
```

Forwarding Refs

jsx

```
const FancyButton = forwardRef((props, ref) => {
  return (
    <button ref={ref} className="fancy-button">
      {props.children}
    </button>
  );
});

function Parent() {
  const buttonRef = useRef();

  useEffect(() => {
    buttonRef.current.focus();
  }, []);

  return <FancyButton ref={buttonRef}>Click me</FancyButton>;
}
```

Common Ref Use Cases

Focus Management

jsx

```
function LoginForm() {
  const emailRef = useRef();
  const passwordRef = useRef();

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!emailRef.current.value) {
      emailRef.current.focus();
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input ref={emailRef} type="email" />
      <input ref={passwordRef} type="password" />
    </form>
  );
}
```

Media Playback

jsx

```
function VideoPlayer({ src }) {  
  const videoRef = useRef();  
  
  const play = () => videoRef.current.play();  
  const pause = () => videoRef.current.pause();  
  
  return (  
    <>  
      <video ref={videoRef} src={src} />  
      <button onClick={play}>Play</button>  
      <button onClick={pause}>Pause</button>  
    </>  
  );  
}
```

Text Selection

jsx

```
function TextSelector() {  
  const textRef = useRef();  
  
  const selectText = () => {  
    textRef.current.select();  
  };  
  
  return <input ref={textRef} value="Select me" readOnly />;  
}
```

Measuring DOM Elements

jsx

```
function MeasureExample() {
  const [height, setHeight] = useState(0);
  const ref = useRef();

  useEffect(() => {
    setHeight(ref.current.clientHeight);
  }, []);

  return (
    <div ref={ref}>
      My height is: {height}px
    </div>
  );
}
```

8. SIDE EFFECTS & LIFECYCLE

useEffect Complete Guide

Dependency Array Patterns

jsx

```
useEffect(() => {
  console.log('Runs after every render');
});

useEffect(() => {
  console.log('Runs once');
}, []);

useEffect(() => {
  console.log('Count changed:', count);
}, [count]);

useEffect(() => {
  console.log('Either count or name changed');
}, [count, name]);
```

Cleanup Function

jsx

```
useEffect(() => {  
  
  const subscription = api.subscribe();  
  const timer = setInterval(() => {}, 1000);  
  
  return () => {  
    subscription.unsubscribe();  
    clearInterval(timer);  
  };  
}, []);
```

Lifecycle Mapping (Class vs Functional)

Common Side Effects

Data Fetching

jsx


```
function DataFetcher() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let isMounted = true;

    async function fetchData() {
      try {
        setLoading(true);
        const response = await
fetch('https://api.example.com/data');
        const json = await response.json();

        if (isMounted) {
          setData(json);
          setError(null);
        }
      } catch (err) {
        if (isMounted) {
          setError(err.message);
        }
      } finally {
        if (isMounted) {
          setLoading(false);
        }
      }
    }

    fetchData();

    return () => {
      isMounted = false;
    };
  }, []);

  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error}</div>;
  return <div>{JSON.stringify(data)}</div>;
}
```

Subscriptions

jsx

```
function EventListener() {
  useEffect(() => {
    const handleResize = () => {
      console.log(window.innerWidth);
    };

    window.addEventListener('resize', handleResize);

    return () => {
      window.removeEventListener('resize', handleResize);
    };
  }, []);

  return <div>Resize the window</div>;
}
```

Timers

jsx

```
function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);

    return () => clearInterval(interval);
  }, []);

  return <div>{seconds} seconds elapsed</div>;
}
```

DOM Manipulation

jsx

```
function TitleUpdater() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `You clicked ${count} times`;
  }, [count]);

  return <button onClick={() => setCount(c => c + 1)}>Click</button>;
}
```

Common useEffect Mistakes

✗ Missing Dependencies

jsx

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      setCount(count + 1);
    }, 1000);
    return () => clearInterval(id);
  }, []);

  setCount(prev => prev + 1);
}
```

✗ Infinite Loops

jsx

```
useEffect(() => {
  fetchData({ id: userId });
}, [{ id: userId }]);

useEffect(() => {
  fetchData({ id: userId });
}, [userId]);
```

9. ADVANCED PATTERNS

Higher-Order Components (HOC)

jsx

```
function withLogging(WrappedComponent) {
  return function WithLogging(props) {
    useEffect(() => {
      console.log(`Component ${WrappedComponent.name}
mounted`);
      return () => console.log(`Component
${WrappedComponent.name} unmounted`);
    }, []);

    return <WrappedComponent {...props} />;
  };
}

const UserPageWithLogging = withLogging(UserPage);
```

Authentication HOC

jsx

```
function withAuth(WrappedComponent) {
  return function WithAuth(props) {
    const { user } = useContext(AuthContext);

    if (!user) {
      return <LoginPage />;
    }

    return <WrappedComponent {...props} user={user} />;
  };
}

const ProtectedProfile = withAuth(Profile);
```

Render Props Pattern

jsx

```
class MouseTracker extends React.Component {
  state = { x: 0, y: 0 };

  handleMouseMove = (e) => {
    this.setState({ x: e.clientX, y: e.clientY });
  };

  render() {
    return (
      <div onMouseMove={this.handleMouseMove}>
        {this.props.render(this.state)}
      </div>
    );
  }
}

<MouseTracker render={({ x, y }) => (
  <h1>The mouse is at ({x}, {y})</h1>
)} />

<MouseTracker>
  {({ x, y }) => <h1>Position: {x}, {y}</h1>}
</MouseTracker>
```

Compound Components

jsx

```
function Toggle({ children }) {
  const [on, setOn] = useState(false);

  const toggle = () => setOn(!on);

  return (
    <ToggleContext.Provider value={{ on, toggle }}>
      {children}
    </ToggleContext.Provider>
  );
}

Toggle.On = ({ children }) => {
  const { on } = useContext(ToggleContext);
  return on ? children : null;
};

Toggle.Off = ({ children }) => {
  const { on } = useContext(ToggleContext);
  return on ? null : children;
};

Toggle.Button = ({ children }) => {
  const { toggle } = useContext(ToggleContext);
  return <button onClick={toggle}>{children}</button>;
};

<Toggle>
  <Toggle.On>The button is on</Toggle.On>
  <Toggle.Off>The button is off</Toggle.Off>
  <Toggle.Button>Toggle</Toggle.Button>
</Toggle>
```

Custom Hooks (Reusable Logic)

jsx

```
function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.log(error);
      return initialValue;
    }
  });

  const setValue = (value) => {
    try {
      const valueToStore = value instanceof Function ?
value(storedValue) : value;
      setStoredValue(valueToStore);
      localStorage.setItem(key, JSON.stringify(valueToStore));
    } catch (error) {
      console.log(error);
    }
  };

  return [storedValue, setValue];
}

function useWindowSize() {
  const [windowSize, setWindowSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    const handleResize = () => {
      setWindowSize({
        width: window.innerWidth,
        height: window.innerHeight,
      });
    };

    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize',
handleResize);
  }, []);

  return windowSize;
}
```

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let isMounted = true;

    const fetchData = async () => {
      try {
        setLoading(true);
        const response = await fetch(url);
        const json = await response.json();

        if (isMounted) {
          setData(json);
          setError(null);
        }
      } catch (err) {
        if (isMounted) setError(err);
      } finally {
        if (isMounted) setLoading(false);
      }
    };

    fetchData();

    return () => {
      isMounted = false;
    };
  }, [url]);

  return { data, loading, error };
}
```

Portal Pattern

jsx


```
function Modal({ children, isOpen, onClose }) {
  if (!isOpen) return null;

  return createPortal(
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal-content" onClick={e =>
e.stopPropagation()}>
        {children}
        <button onClick={onClose}>Close</button>
      </div>
    </div>,
    document.getElementById('portal-root')
  );
}

function App() {
  const [isOpen, setIsOpen] = useState(false);

  return (
    <div>
      <button onClick={() => setIsOpen(true)}>Open
Modal</button>
      <Modal isOpen={isOpen} onClose={() => setIsOpen(false)}>
        <h2>Modal Title</h2>
        <p>This content is portaled to body</p>
      </Modal>
    </div>
  );
}
```

Error Boundaries (Class Components Only)

jsx

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  componentDidCatch(error, errorInfo) {
    console.log('Error caught:', error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return (
        <div>
          <h2>Something went wrong</h2>
          <details>{this.state.error?.toString()}</details>
          <button onClick={() => this.setState({ hasError:
false })}>
            Try again
          </button>
        </div>
      );
    }

    return this.props.children;
  }
}

<ErrorBoundary>
  <BuggyComponent />
</ErrorBoundary>
```

10. REACT 18+ FEATURES

Automatic Batching

jsx

```
setTimeout(() => {  
  setCount(c => c + 1);  
  setFlag(f => !f);  
}, 1000);  
  
setTimeout(() => {  
  setCount(c => c + 1);  
  setFlag(f => !f);  
}, 1000);
```

Transitions

jsx

```
import { useTransition, startTransition } from 'react';  
  
function SearchPage() {  
  const [isPending, startTransition] = useTransition();  
  const [query, setQuery] = useState('');  
  const [results, setResults] = useState([]);  
  
  const handleChange = (e) => {  
    const value = e.target.value;  
    setQuery(value);  
  
    startTransition(() => {  
      const filtered = filterResults(value);  
      setResults(filtered);  
    });  
  };  
  
  return (  
    <div>  
      <input value={query} onChange={handleChange} />  
      {isPending && <Spinner />}  
      {results.map(result => <Result key={result.id}  
        {...result} />)}  
    </div>  
  );  
}
```

Suspense for Data Fetching

jsx

```
function fetchUser(id) {
  let status = 'pending';
  let result;
  const suspender = fetch(`/api/users/${id}`).then(
    r => r.json(),
    e => { throw e; }
  );

  return {
    read() {
      if (status === 'pending') throw suspender;
      if (status === 'error') throw result;
      return result;
    }
  };
}

function UserProfile({ userId }) {
  const user = fetchUser(userId).read();
  return <div>{user.name}</div>;
}

function App() {
  return (
    <Suspense fallback={<div>Loading user...</div>}>
      <UserProfile userId={1} />
    </Suspense>
  );
}
```

Strict Mode (React 18 Enhancements)

jsx

```
<StrictMode>
  <App />
</StrictMode>
```

New Hooks (React 18)

jsx

```
function NameField() {
  const id = useId();
  return (
    <>
      <label htmlFor={id}>Name</label>
      <input id={id} type="text" />
    </>
  );
}

const store = createStore();
const state = useSyncExternalStore(
  store.subscribe,
  store.getState,
  () => initialState
);

useInsertionEffect(() => {

  const style = document.createElement('style');
  style.textContent = `.dynamic { color: red; }`;
  document.head.appendChild(style);
  return () => style.remove();
}, []);
```

11. ECOSYSTEM & TOOLS

State Management Libraries

Redux

jsx

```

import { configureStore, createSlice } from
 '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; },
    decrement: (state) => { state.value -= 1; }
  }
});

const store = configureStore({
  reducer: { counter: counterSlice.reducer }
});

import { useSelector, useDispatch } from 'react-redux';

function Counter() {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();

  return (
    <>
      <button onClick={() => dispatch(increment())}>+</button>
      <span>{count}</span>
    </>
  );
}

```

Zustand (Simpler alternative)

jsx

```

import create from 'zustand';

const useStore = create((set) => ({
  bears: 0,
  increasePopulation: () => set(state => ({ bears: state.bears
+ 1 })),
  removeAllBears: () => set({ bears: 0 })
}));

function BearCounter() {

```

```

const express = require("express"); const app = express(); const mongoose =
require("mongoose"); const Listing = require("./models/listing.js"); const path =
require("path"); const methodOverride = require("method-override"); const
ejsMate = require("ejs-mate"); const PORT = 8080; const MONGO_URL =
"mongodb://127.0.0.1:27017/wanderlust"; //
===== // MongoDB Connection //
===== main() .then() => {
console.log(" Connected to MongoDB"); }) .catch((err) => { console.log("
Database Connection Error:", err); }); async function main() { await
mongoose.connect(MONGO_URL); } //
===== // View Engine Setup //
===== app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views")); app.engine("ejs", ejsMate); //
===== // Middleware //
===== app.use(express.urlencoded({
extended: true })); app.use(express.json());
app.use(methodOverride("_method"));
app.use(express.static(path.join(__dirname, "public"))); //
===== // Root Route //
===== app.get("/", (req, res) => {
res.send("Welcome to Wanderlust!"); }); //
===== // INDEX Route - Show All
Listings // ===== app.get("/listings",
async (req, res) => { try { const allListings = await Listing.find({});
res.render("listings/index.ejs", { allListings }); } catch (err) { console.log(err);
res.send("Error fetching listings"); } }); //
===== // NEW Route - Form to
Create Listing // =====
app.get("/listings/new", (req, res) => { res.render("listings/new.ejs"); }); //
===== // SHOW Route - Show Single
Listing // =====
app.get("/listings/:id", async (req, res) => { try { const { id } = req.params; const
listing = await Listing.findById(id); if (!listing) { return res.send("Listing not
found"); } res.render("listings/show.ejs", { listing }); } catch (err) {
console.log(err); res.send("Error loading listing details"); } }); //
===== // CREATE Route - Add New
Listing // ===== app.post("/listings",
async (req, res) => { try { const newListing = new Listing(req.body.listing); await
newListing.save(); console.log(" New Listing Created"); res.redirect("/listings"); }
catch (err) { console.log(err); res.send("Error creating listing"); } }); //
===== // EDIT Route - Edit Form //
===== app.get("/listings/:id/edit",

```

```

async (req, res) => { try { const { id } = req.params; const listing = await
Listing.findById(id); if (!listing) { return res.send("Listing not found"); }
res.render("listings/edit.ejs", { listing }); } catch (err) { console.log(err);
res.send("Error loading edit form"); } }); //
===== // UPDATE Route - Update
Listing // =====
app.put("/listings/:id", async (req, res) => { try { const { id } = req.params; await
Listing.findByIdAndUpdate(id, { ...req.body.listing, }); console.log(" Listing
Updated"); res.redirect(`/listings/${id}`); } catch (err) { console.log(err);
res.send("Error updating listing"); } }); //
===== // DELETE Route - Delete
Listing // =====
app.delete("/listings/:id", async (req, res) => { try { const { id } = req.params;
const deletedListing = await Listing.findByIdAndDelete(id); console.log("
Deleted Listing:", deletedListing); res.redirect("/listings"); } catch (err) {
console.log(err); res.send("Error deleting listing"); } }); //
===== // Test Route - Sample Data //
===== app.get("/testListing", async
(req, res) => { try { const sampleListing = new Listing({ title: "My New Villa",
description: "By the beach", price: 1200, location: "Calangute, Goa", country:
"India", image: "https://images.unsplash.com/photo-1505693416388-
ac5ce068fe85", }); await sampleListing.save(); console.log(" Sample Listing
Saved"); res.send("Sample listing created successfully"); } catch (err) {
console.log(err); res.send("Error creating sample listing"); } }); //
===== // Server Start //
===== app.listen(PORT, () => {
console.log(` Server is running on port ${PORT}`); }); explain every word easy
way

```

I'll explain this **Node.js + Express + MongoDB** backend code **line by line** in the easiest way possible, like you're learning it for the first time.



COMPLETE LINE-BY-LINE EXPLANATION

PART 1: IMPORTING LIBRARIES (Lines 1-6)

javascript

```
const express = require("express");
```

Easy meaning: Bring Express framework into your project. Express helps create web servers easily.

javascript

```
const app = express();
```

Easy meaning: Create an application object from Express. **app** is your main tool to handle requests (like GET, POST).

javascript

```
const mongoose = require("mongoose");
```

Easy meaning: Bring Mongoose - a tool to connect and work with MongoDB database.

javascript

```
const Listing = require("../models/listing.js");
```

Easy meaning: Import the **Listing** model (blueprint for your data). This model defines what a "listing" looks like in your database (title, price, location, etc.).

javascript

```
const path = require("path");
```

Easy meaning: Bring Node.js **path** module - helps work with file and folder paths.

javascript

```
const methodOverride = require("method-override");
```

Easy meaning: Bring **method-override** - allows browsers to send PUT/DELETE requests (HTML forms only support GET/POST normally).

javascript

```
const ejsMate = require("ejs-mate");
```

Easy meaning: Bring **ejs-mate** - makes EJS templates better with layouts and partials.

PART 2: CONFIGURATION (Lines 8-9)

javascript

```
const PORT = 8080;
```

Easy meaning: Set the port number (like a door number) where your server will listen. **8080** means **http://localhost:8080**

javascript

```
const MONGO_URL = "mongodb://127.0.0.1:27017/wanderlust";
```

Easy meaning: Tell your app where MongoDB is running.

- **127.0.0.1** = your own computer (localhost)
- **27017** = default MongoDB port
- **wanderlust** = database name

PART 3: MONGODB CONNECTION (Lines 13-21)

javascript

```
main()
  .then(() => {
    console.log(" Connected to MongoDB");
  })
  .catch((err) => {
    console.log(" Database Connection Error:", err);
  });
```

Easy meaning: Try to connect to MongoDB. If successful → show success message. If error → show error message.

javascript

```
async function main() {  
  await mongoose.connect(MONGO_URL);  
}
```

Easy meaning: Define a function that connects to MongoDB. `async/await` means "wait until connected before moving on."

PART 4: VIEW ENGINE SETUP (Lines 24-27)

javascript

```
app.set("view engine", "ejs");
```

Easy meaning: Tell Express to use EJS (Embedded JavaScript) for rendering HTML pages.

javascript

```
app.set("views", path.join(__dirname, "views"));
```

Easy meaning: Tell Express that all your template files are in the `views` folder.

javascript

```
app.engine("ejs", ejsMate);
```

Easy meaning: Tell Express to use `ejsMate` for handling EJS files (gives you layouts and better template features).

PART 5: MIDDLEWARE (Lines 30-33)

javascript

```
app.use(express.urlencoded({ extended: true }));
```

Easy meaning: Parse incoming form data (from HTML forms) so you can read it in `req.body`.

javascript

```
app.use(express.json());
```

Easy meaning: Parse JSON data (from APIs/mobile apps) so you can read it in `req.body`.

javascript

```
app.use(methodOverride("_method"));
```

Easy meaning: Override HTTP methods. If someone sends `?_method=PUT`, treat it as a PUT request (because HTML forms can only send GET/POST).

javascript

```
app.use(express.static(path.join(__dirname, "public")));
```

Easy meaning: Serve static files (CSS, JS, images) from the `public` folder.

PART 6: ROOT ROUTE (Lines 37-39)

javascript

```
app.get("/", (req, res) => {  
  res.send("Welcome to Wanderlust!");  
});
```

Easy meaning: When someone visits `http://localhost:8080/`, send back "Welcome to Wanderlust!" as response.

PART 7: INDEX ROUTE - SHOW ALL LISTINGS (Lines 42-50)

javascript

```
app.get("/listings", async (req, res) => {
```

Easy meaning: When user visits `/listings`, run this async function.

javascript

```
try {  
  const allListings = await Listing.find({});
```

Easy meaning: Try to fetch ALL documents from the `listings` collection in MongoDB. `find({})` means "give me everything".

javascript

```
res.render("listings/index.ejs", { allListings });
```

Easy meaning: Render the `index.ejs` template and pass `allListings` data to it.

javascript

```
} catch (err) {  
  console.log(err);  
  res.send("Error fetching listings");
```

Easy meaning: If something goes wrong, log the error and show error message to user.

PART 8: NEW ROUTE - FORM TO CREATE (Lines 53-55)

javascript

```
app.get("/listings/new", (req, res) => {  
  res.render("listings/new.ejs");  
});
```

Easy meaning: When user visits `/listings/new`, show the form (`new.ejs`) where they can enter listing details.

PART 9: SHOW ROUTE - SINGLE LISTING (Lines 58-70)

javascript

```
app.get("/listings/:id", async (req, res) => {
```

Easy meaning: When user visits `/listings/someID`, run this. `:id` is a dynamic parameter.

javascript

```
try {  
  const { id } = req.params;
```

Easy meaning: Extract the **id** from the URL. Example: **/listings/123** → **id = 123**

javascript

```
const listing = await Listing.findById(id);
```

Easy meaning: Search MongoDB for a listing with that specific ID.

javascript

```
if (!listing) {  
  return res.send("Listing not found");  
}
```

Easy meaning: If no listing found with that ID, show "Listing not found" message.

javascript

```
res.render("listings/show.ejs", { listing });
```

Easy meaning: Show the **show.ejs** template with the listing data.

javascript

```
} catch (err) {  
  console.log(err);  
  res.send("Error loading listing details");
```

Easy meaning: If error occurs, log it and show error message.

PART 10: CREATE ROUTE - ADD NEW LISTING (Lines 73-83)

javascript

```
app.post("/listings", async (req, res) => {
```

Easy meaning: When user submits form (POST request) to `/listings`, run this.

javascript

```
try {  
  const newListing = new Listing(req.body.listing);
```

Easy meaning: Create a new Listing object using the data sent from the form.

Example `req.body.listing`:

javascript

```
{  
  title: "Beach House",  
  price: 200,  
  location: "Goa",  
  country: "India"  
}
```

javascript

```
await newListing.save();
```

Easy meaning: Save this new listing to MongoDB database.

javascript

```
console.log(" New Listing Created");  
res.redirect("/listings");
```

Easy meaning: Log success message, then redirect user to the listings page.

PART 11: EDIT ROUTE - SHOW EDIT FORM (Lines 86-97)

javascript

```
app.get("/listings/:id/edit", async (req, res) => {
```

Easy meaning: When user visits `/listings/123/edit`, show edit form.

javascript

```
try {  
  const { id } = req.params;  
  const listing = await Listing.findById(id);
```

Easy meaning: Get ID from URL, find that listing in database.

javascript

```
if (!listing) {  
  return res.send("Listing not found");  
}
```

Easy meaning: If no listing found, show error.

javascript

```
res.render("listings/edit.ejs", { listing });
```

Easy meaning: Show edit form with current listing data pre-filled.

PART 12: UPDATE ROUTE - MODIFY LISTING (Lines 100-112)

javascript

```
app.put("/listings/:id", async (req, res) => {
```

Easy meaning: When user submits edit form (with `_method=PUT`), update the listing.

javascript

```
try {  
  const { id } = req.params;
```

Easy meaning: Get the ID from URL.

javascript


```
await Listing.findByIdAndUpdate(id, {  
  ...req.body.listing,  
});
```

Easy meaning: Find listing by ID and update it with new data from form.

- `...` = spread operator (copy all properties)

javascript

```
console.log(" Listing Updated");  
res.redirect(`/listings/${id}`);
```

Easy meaning: Log success, redirect to the updated listing's detail page.

PART 13: DELETE ROUTE - REMOVE LISTING (Lines 115-126)

javascript

```
app.delete("/listings/:id", async (req, res) => {
```

Easy meaning: Handle delete request (from form with `_method=DELETE`).

javascript

```
try {  
  const { id } = req.params;
```

Easy meaning: Get ID from URL.

javascript

```
const deletedListing = await  
Listing.findByIdAndDelete(id);
```

Easy meaning: Find listing by ID and delete it from database.

javascript

```
console.log(" Deleted Listing:", deletedListing);  
res.redirect("/listings");
```

Easy meaning: Log what was deleted, then redirect to listings page.

PART 14: TEST ROUTE - SEED DATA (Lines 129-146)

javascript

```
app.get("/testListing", async (req, res) => {
```

Easy meaning: Helper route to quickly create sample data (for testing).

javascript

```
try {  
  const sampleListing = new Listing({  
    title: "My New Villa",  
    description: "By the beach",  
    price: 1200,  
    location: "Calangute, Goa",  
    country: "India",  
    image: "https://images.unsplash.com/photo-1505693416388-ac5ce068fe85",  
  });
```

Easy meaning: Create a sample listing object.

javascript

```
await sampleListing.save();  
console.log(" Sample Listing Saved");  
res.send("Sample listing created successfully");
```

****Easy meaning**

Failed to load this message

AI-generated, for reference only

state, prop drilling, spa let const var useeffect

complete react interview questions basics to advanced

map, reduce, filter easy way

output result code code type questions and ans

react output result code code type questions and ans

react all concept explain it

```
const express = require("express"); const app = express(); const mongoose =
require("mongoose"); const Listing = require("./models/listing.js"); const path =
require("path"); const methodOverride = require("method-override"); const
ejsMate = require("ejs-mate"); const PORT = 8080; const MONGO_URL =
"mongodb://127.0.0.1:27017/wanderlust"; //
===== // MongoDB Connection //
===== main() .then() => {
console.log(" Connected to MongoDB"); }) .catch((err) => { console.log("
Database Connection Error:", err); }); async function main() { await
mongoose.connect(MONGO_URL); } //
===== // View Engine Setup //
===== app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views")); app.engine("ejs", ejsMate); //
===== // Middleware //
===== app.use(express.urlencoded({
extended: true })); app.use(express.json());
app.use(methodOverride("_method"));
app.use(express.static(path.join(__dirname, "public"))); //
===== // Root Route //
===== app.get("/", (req, res) => {
res.send("Welcome to Wanderlust!"); }); //
===== // INDEX Route - Show All
Listings // ===== app.get("/listings",
async (req, res) => { try { const allListings = await Listing.find({});
res.render("listings/index.ejs", { allListings }); } catch (err) { console.log(err);
res.send("Error fetching listings"); } }); //
===== // NEW Route - Form to
Create Listing // =====
app.get("/listings/new", (req, res) => { res.render("listings/new.ejs"); }); //
===== // SHOW Route - Show Single
Listing // =====
app.get("/listings/:id", async (req, res) => { try { const { id } = req.params; const
listing = await Listing.findById(id); if (!listing) { return res.send("Listing not
found"); } res.render("listings/show.ejs", { listing }); } catch (err) {
console.log(err); res.send("Error loading listing details"); } }); //
===== // CREATE Route - Add New
Listing // ===== app.post("/listings",
```

```

async (req, res) => { try { const newListing = new Listing(req.body.listing); await
newListing.save(); console.log(" New Listing Created"); res.redirect("/listings"); }
catch (err) { console.log(err); res.send("Error creating listing"); } }); //
===== // EDIT Route - Edit Form //
===== app.get("/listings/:id/edit",
async (req, res) => { try { const { id } = req.params; const listing = await
Listing.findById(id); if (!listing) { return res.send("Listing not found"); }
res.render("listings/edit.ejs", { listing }); } catch (err) { console.log(err);
res.send("Error loading edit form"); } }); //
===== // UPDATE Route - Update
Listing // =====
app.put("/listings/:id", async (req, res) => { try { const { id } = req.params; await
Listing.findByIdAndUpdate(id, { ...req.body.listing, }); console.log(" Listing
Updated"); res.redirect(`/listings/${id}`); } catch (err) { console.log(err);
res.send("Error updating listing"); } }); //
===== // DELETE Route - Delete
Listing // =====
app.delete("/listings/:id", async (req, res) => { try { const { id } = req.params;
const deletedListing = await Listing.findByIdAndDelete(id); console.log("
Deleted Listing:", deletedListing); res.redirect("/listings"); } catch (err) {
console.log(err); res.send("Error deleting listing"); } }); //
===== // Test Route - Sample Data //
===== app.get("/testListing", async
(req, res) => { try { const sampleListing = new Listing({ title: "My New Villa",
description: "By the beach", price: 1200, location: "Calangute, Goa", country:
"India", image: "https://images.unsplash.com/photo-1505693416388-
ac5ce068fe85", }); await sampleListing.save(); console.log(" Sample Listing
Saved"); res.send("Sample listing created successfully"); } catch (err) {
console.log(err); res.send("Error creating sample listing"); } }); //
===== // Server Start //
===== app.listen(PORT, () => {
console.log(` Server is running on port ${PORT}`); }); explain every word easy
way

```